00 01 02 03 04 **05** 06 07 08 09 0A 0B 0C 0D

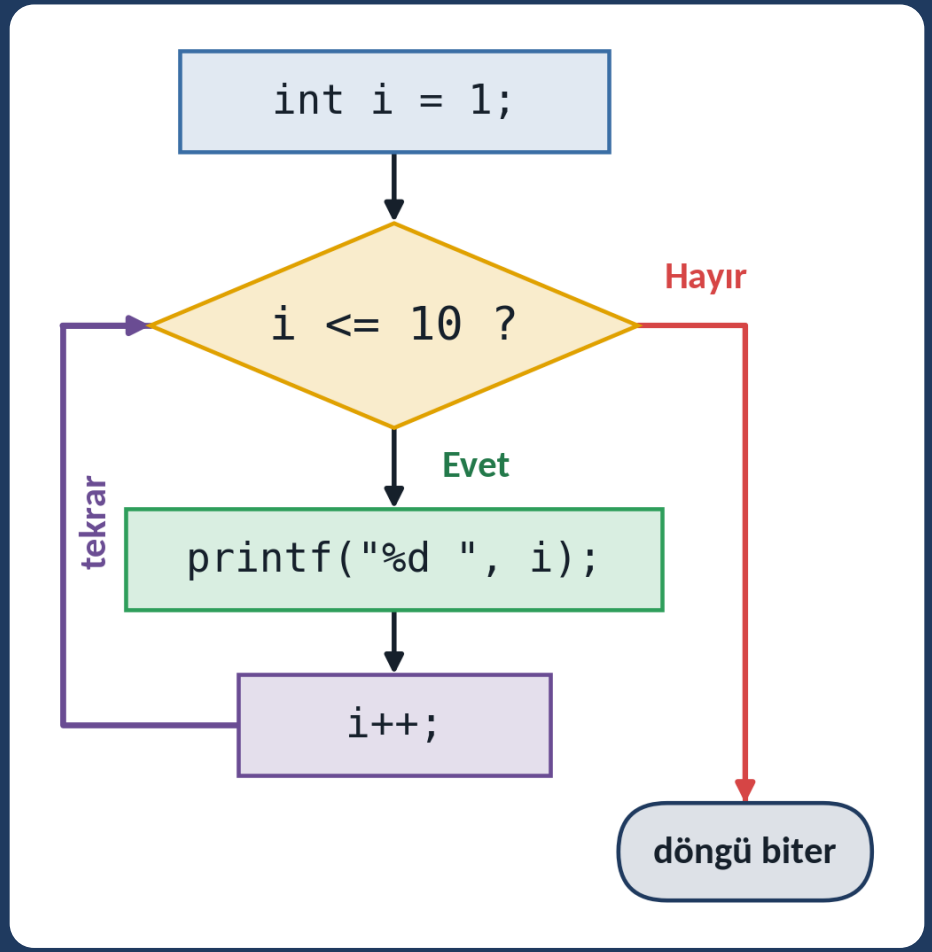
ALGORİTMA VE PROGRAMLAMA – I

HAFTA 06

Döngüler I: Tekrarın Mantığı ve Temel Döngü Desenleri

while, do-while ve for · sayaç ve nöbetçi değer · toplayıcıdan basamak ayrıştırmaya döngü desenleri · döngünün makine kodu

MODÜL III · KONTROL AKIŞI



Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Güz

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1	H2	H3	H4	H5	H6	H7
Bilgisayar ve Program	Algoritmik Düşünme	Değişkenler ve Veri Tipleri	Operatörler ve İfadeler	Karar Yapıları	**Döngüler I**	Döngüler II + Ara Proje
0x00	0x01	0x02	0x03	0x04	0x05	0x06

H8	H9	H10	H11	H12	H13	H14
Fonksiyonlar I	Kapsam, Yığın, Özyineleme	Diziler I	Diziler II: Matrisler	İşaretçiler ve Bellek	Karakter Dizileri	Yapılar, Dosyalar, Final
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 · Final projesi: H14

GERİ ÇAĞIRMA · H5

Geçen haftadan · S1: Bu program ne yazdırır?

quiz/H06_Q01_switch_duser.c · satır 10–24

```
10      int secim = 2;
11      switch (secim) {
12      case 1:
13          printf("Bir ");
14          break;
15      case 2:
16          printf("İki ");
17          /* fall through - break yok: aşağıya düşer */
18      case 3:
19          printf("Üç ");
20          break;
21      default:
22          printf("Diğer ");
23      }
24      printf("\n");
```

A İki

B İki Üç

C İki Üç Diğer

D Derleme hatası: `break` eksik

TAHMİN ET

Geçen haftadan · S2 ve S3

S2 (bileşik koşul): `puan = 105` için hangi ileti yazılır?

Koşulu `!` kullanmadan yazın (De Morgan).

quiz/H06_Q02_de_morgan.c · satır 10–15

```c
10      int puan = 105;
11      if (!(puan >= 0 && puan <= 100)) {
12          printf("Geçersiz puan: %d\n", puan);
13      } else {
14          printf("Geçerli puan: %d\n", puan);
15      }
```

S3 (tek şans): Kullanıcı `150` girerse ne olur?

Yeniden sorabilmek için hangi yapı gerekir?

İpucu: kaç kez sorulacağını önceden biliyor muyuz?

quiz/H06_Q03_tek_sans.c · satır 11–17

```c
11      int puan;
12      printf("Puan (0-100): ");
13      if (scanf("%d", &puan) != 1 || puan < 0 || puan > 100) {
14          printf("Geçersiz puan; program sonlanıyor.\n");
15          return 1;
16      }
17      printf("Puan kabul edildi: %d\n", puan);
```

Yanıtlar: gerçek çıktılar

S1 · B

GERÇEK ÇIKTI

```
$ ./q01
İki Üç
```

- `case 2`'de `break` yok → **düşme** (fall-through)
- `case 3`'teki `break`'te durur

S2 · Geçersiz

GERÇEK ÇIKTI

```
$ ./q02
Geçersiz puan: 105
```

- `!(p >= 0 && p <= 100)`
- ≡ `p < 0 || p > 100`

S3 · tek şans

GERÇEK ÇIKTI

```
$ ./q03
Puan (0-100): 150
Geçersiz puan; program sonlanıyor.
```

- Program **biter**; yeniden sormaz
- Gereken: "geçerli olana kadar **tekrar**" → döngü

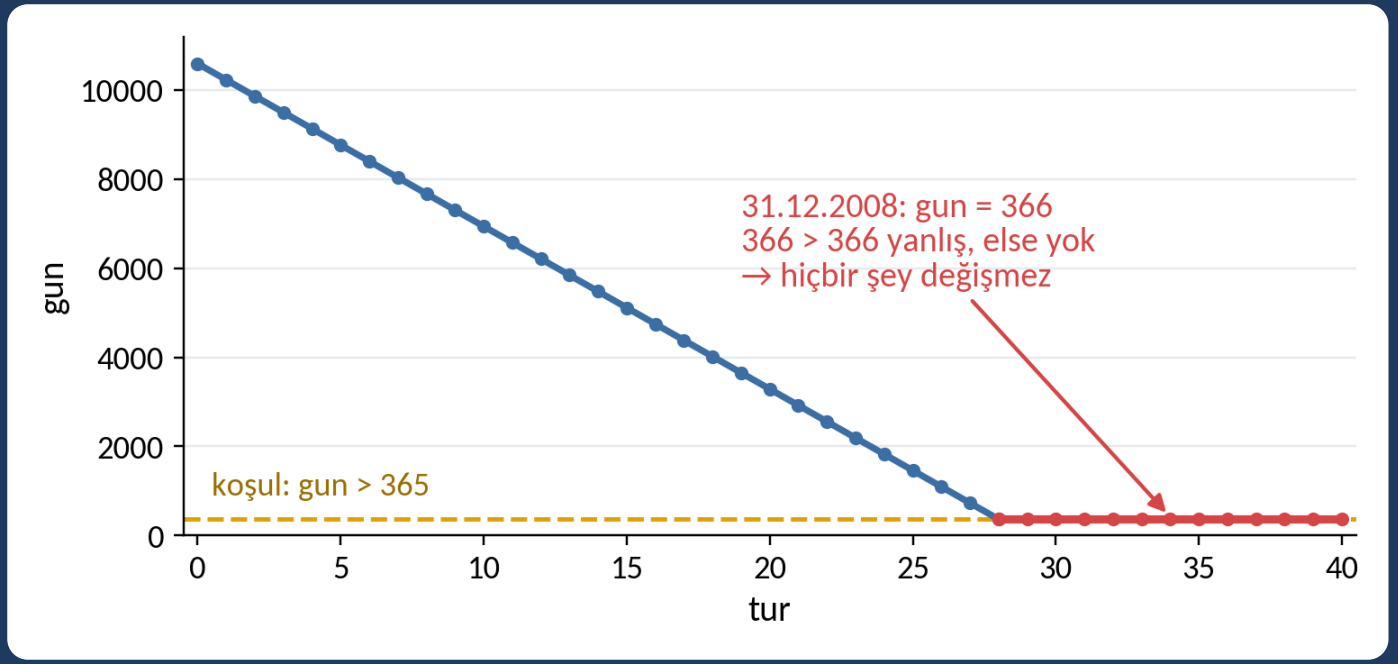
Bu hafta: aynı işi **bir koşul doğru olduğu sürece** tekrar et → `while`, `do-while`, `for`

GERÇEK DÜNYADAN

31.12.2008

31 Aralık 2008: Zune 30’lar neden açılmadı?

- 30 GB’lık ilk nesil Zune’lar açılışta **takılıp kaldı**
- Neden: 1.1.1980’den geçen günlerden yılı bulan bir **döngü**; 2008 artık yıl
- 366. gün: `gun > 365` doğru, `gun > 366` yanlış → **hiçbir değişken değişmez**
- Önerilen çözüm: pil bitsin, 1 Ocak 2009’da şarj edip yeniden aç
- Döngünün eğitsel yeniden kurgusu: **Hata Avcısı 7**



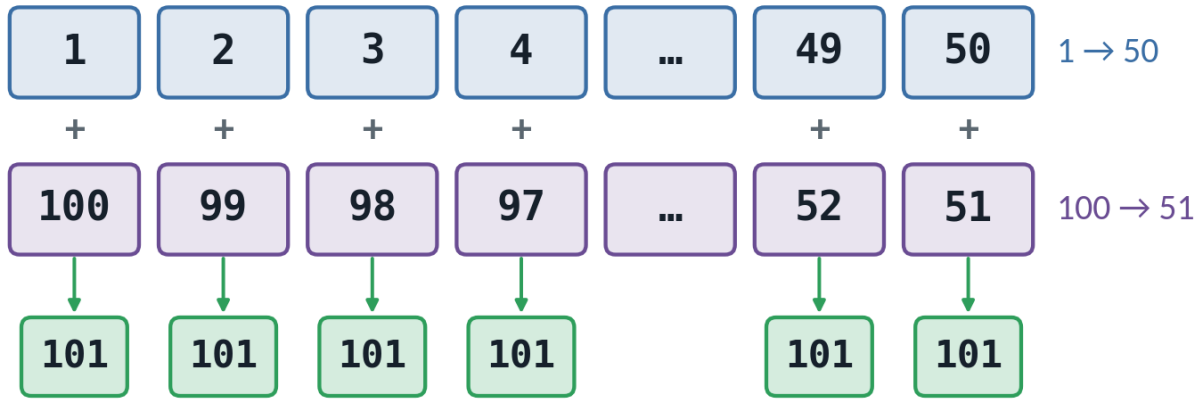
gun değişkeni tur tur (gösterim sürümünün gerçek çıktısıyla sağlandı): 28. turdan sonra hep 366

Coldewey (2008), TechCrunch · “Microsoft says Zune players working again” (2009), NBC News · Hayes (2009), bit-player

BÜYÜK SORU

Haftanın Büyük Sorusu

Bilgisayar aynı işi milyarlarca kez nasıl yorulmadan yapar ve ne zaman durması gerektiğini nereden bilir?



50 çift × 101 = 5050 · döngü: 100 tur · formül: 3 işlem

Karşı öykü (anlatılagelen): küçük Gauss 1 + … + 100'ü uçlardan eşleştirerek buldu: 50 × 101 = 5050. Kimi zaman en iyi döngü, hiç yazılmayandır.

Yorulmadan?

Bu makinede $1 + \ldots + 10^9$ toplamı: **2,5 s** (iyileştirmesiz). Nasıl?

Ne zaman durur?

Zune'da tek bir günde durmadı. Durmayı kim, neye göre bilir?

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 6.1	while, do-while ve for döngülerinin çalışma mantığını ve farklarını açıklar; probleme uygun döngüyü seçer.
ÖÇ 6.2	Sayaç kontrollü ve nöbetçi değer (sentinel) kontrollü döngüler tasarlar.
ÖÇ 6.3	Biriktirme (toplam/çarpım), sayma, en büyük/en küçük takibi, bayrak ve basamak işleme desenlerini uygular.
ÖÇ 6.4	İzleme tablosuyla döngü davranışını elle yürütür; sonsuz döngü ve bir-fark (off-by-one) hatalarını tespit eder.

01

Tekrarın mantığı: üç döngü

DRY ilkesi · döngünün dört parçası · while, do-while, for · kaç kez döner?

KARŞILAŞTIR

Neden döngü? Kalıbı bir kez yaz

Döngüsüz: 5 kopya

H06_K01_dry_tekrar.c · satır 12–16

```c
12      printf("%d. öğrenci kaydı işlendi\n", 1);
13      printf("%d. öğrenci kaydı işlendi\n", 2);
14      printf("%d. öğrenci kaydı işlendi\n", 3);
15      printf("%d. öğrenci kaydı işlendi\n", 4);
16      printf("%d. öğrenci kaydı işlendi\n", 5);
```

Döngüyle: kalıp bir kez

H06_K01_dry_tekrar.c · satır 21–23

```c
21      for (int i = 1; i <= 5; i++) {
22          printf("%d. öğrenci kaydı işlendi\n", i);
23      }
```

500 kayıt olsaydı?

Solda 500 satır; sağda yine 3 satır.

DRY: Don’t Repeat Yourself

Değişen kısım `i` oldu; bir düzeltme **tek yerde** yapılır.

GERÇEK ÇIKTI: İKİ BLOK AYNI BEŞ SATIRI YAZAR

```
$ ./dry_tekrar
1. öğrenci kaydı işlendi
2. öğrenci kaydı işlendi
3. öğrenci kaydı işlendi
4. öğrenci kaydı işlendi
5. öğrenci kaydı işlendi
---
1. öğrenci kaydı işlendi
2. öğrenci kaydı işlendi
3. öğrenci kaydı işlendi
4. öğrenci kaydı işlendi
5. öğrenci kaydı işlendi
```

KAVRAM

Döngünün anatomisi: dört parça

H06_02_toplam_n.c

```
for (int i = 1; i <= n; i++) {
    toplam += i;
}
```

1. **BAŞLATMA** — yalnızca bir kez, en başta
2. **KOŞUL** — her turun başında sınanır
3. **GÖVDE** — koşul doğruysa çalışır
4. **GÜNCELLEME** — gövdeden SONRA; koşula döner

Çalışma sırası (n = 3):



Koşul, gövdeden bir fazla sınanır: n tur için n + 1 denetim (top = toplam).

Başlatma 1 kez · koşul n + 1 kez · gövde n kez · güncelleme n kez

for başlığı üç parçayı (başlatma, koşul, güncelleme) tek satırda toplar; dördüncü parça gövdedir

while ve do-while: koşul önce mi, sonra mı?

while: koşul ÖNCE → 0 tur olabilir

H06_K02_while_temel.c · satır 11–16

```
11      int sayac = 3;                      /* 1) başlatma    */
12      while (sayac > 0) {                 /* 2) koşul       */
13          printf("%d...\n", sayac);       /* 3) gövde       */
14          sayac--;                        /* 4) güncelleme */
15      }
16      printf("Kalkış! (sayac = %d)\n", sayac);
```

GERÇEK ÇIKTI (TÜM PROGRAM)

```
$ ./while_temel
3...
2...
1...
Kalkış! (sayac = 0)
İkinci döngü 0 kez çalıştı.
```

do-while: koşul SONRA → en az 1 tur

H06_K03_do_while_temel.c · satır 17–21

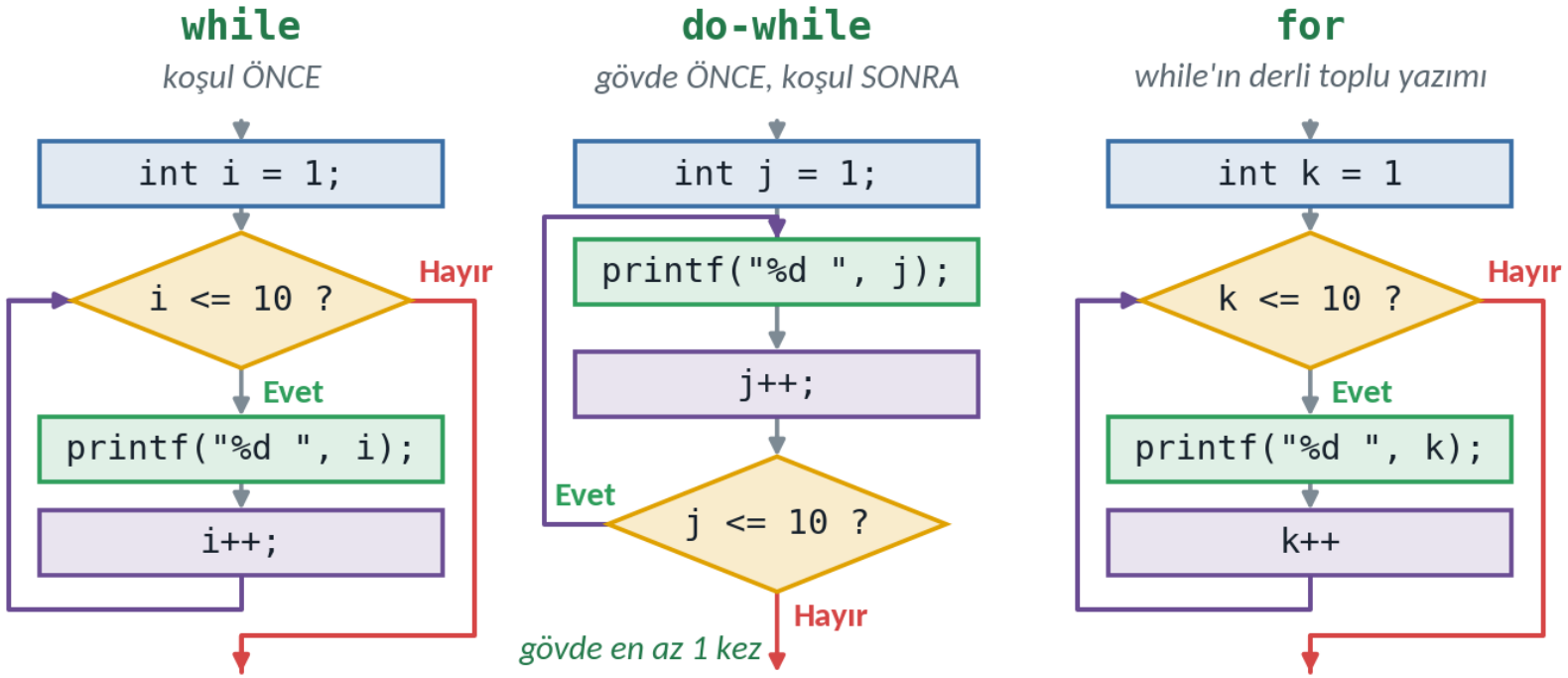
```
17      int j = 10;
18      do {
19          printf("do-while gövdesi: j = %d\n", j);
20          j++;
21      } while (j < 5);                    /* sondaki ; zorunlu */
```

GERÇEK ÇIKTI

```
$ ./do_while_temel
do-while gövdesi: j = 10
Son değerler: i = 10, j = 11
```

Aynı dosyada `while (i < 5)` (i = 10) gövdesi **hiç** çalışmaz; do-while gövdesi **bir kez** çalışır. Sondaki `;` zorunlu.

Üç döngünün akış diyagramı



- while ve for: **karar önce**
- do-while: **gövde önce**, karar sonra
- for: başlatma ve güncelleme **başlıkta**
- Üçü de birbirine **dönüştürülebilir**
- Seçim = okunabilirlik kararı

KOD

for ↔ while dönüşümü

H06_K04_for_while_donusum.c · satır 11–21

```c
    for (int i = 1; i <= 3; i++) {
        printf("for   turu %d\n", i);
    }
    /* burada i yoktur: kapsamı for döngüsüyle sınırlı */

    int k = 1;                          /* başlatma döngüden önceye */
    while (k <= 3) {                    /* koşul aynen              */
        printf("while turu %d\n", k);
        k++;                            /* güncelleme gövdenin sonuna */
    }
    printf("while bitti: k = %d\n", k); /* k hâlâ erişilebilir */
```

GERÇEK ÇIKTI

```
$ ./for_while_donusum
for   turu 1
for   turu 2
for   turu 3
while turu 1
while turu 2
while turu 3
while bitti: k = 4
```

- Başlatma → döngüden **önce**
- Koşul → **aynen**
- Güncelleme → gövdenin **sonuna**
- for'daki `i` döngüden sonra **yok** (kapsam, H9)

KARŞILAŞTIRMA

Hangi döngü?

	while	do-while	for
Koşul ne zaman sınanır?	Gövdeden önce	Gövdeden sonra	Gövdeden önce
En az tur sayısı	0	1	0
Başlatma	Döngüden önce	Döngüden önce	Başlıkta (1. parça)
Güncelleme	Gövdenin içinde	Gövdenin içinde	Başlıkta (3. parça)
Tipik kullanım	Nöbetçi değer, Öklid	Girdi doğrulama, menü	Sayaç kontrollü: 1..N, tablo
Seçim sorusu	**Tur sayısı belli değil**	**En az bir kez çalışmalı**	**Tur sayısı baştan belli**

TAHMİN ET

Kaç kez döner? `tur++` kaç kez çalışır?

a–d'nin gövdesi aynı: `tur++;` (satır 12); e ve f'nin kodu tam.

a

quiz · H06_Q21_… · satır 11

```
for (int i = 0; i < 10; i++) {
```

? tur

b

quiz · H06_Q22_… · satır 11

```
for (int i = 0; i <= 10; i++) {
```

? tur

c

quiz · H06_Q25_… · satır 11

```
for (int i = 10; i >= 0; i -= 3) {
```

? tur

d

quiz · H06_Q26_… · satır 11

```
for (int i = 5; i < 5; i++) {
```

? tur

e

quiz · H06_Q28_… · satır 11–15

```
int i = 10;
do {
    i++;
    tur++;
} while (i < 5);
```

? tur

f

quiz · H06_Q29_… · satır 12–15

```
for (int i = 0; i < 5; i++);
{
    tur++;
}
```

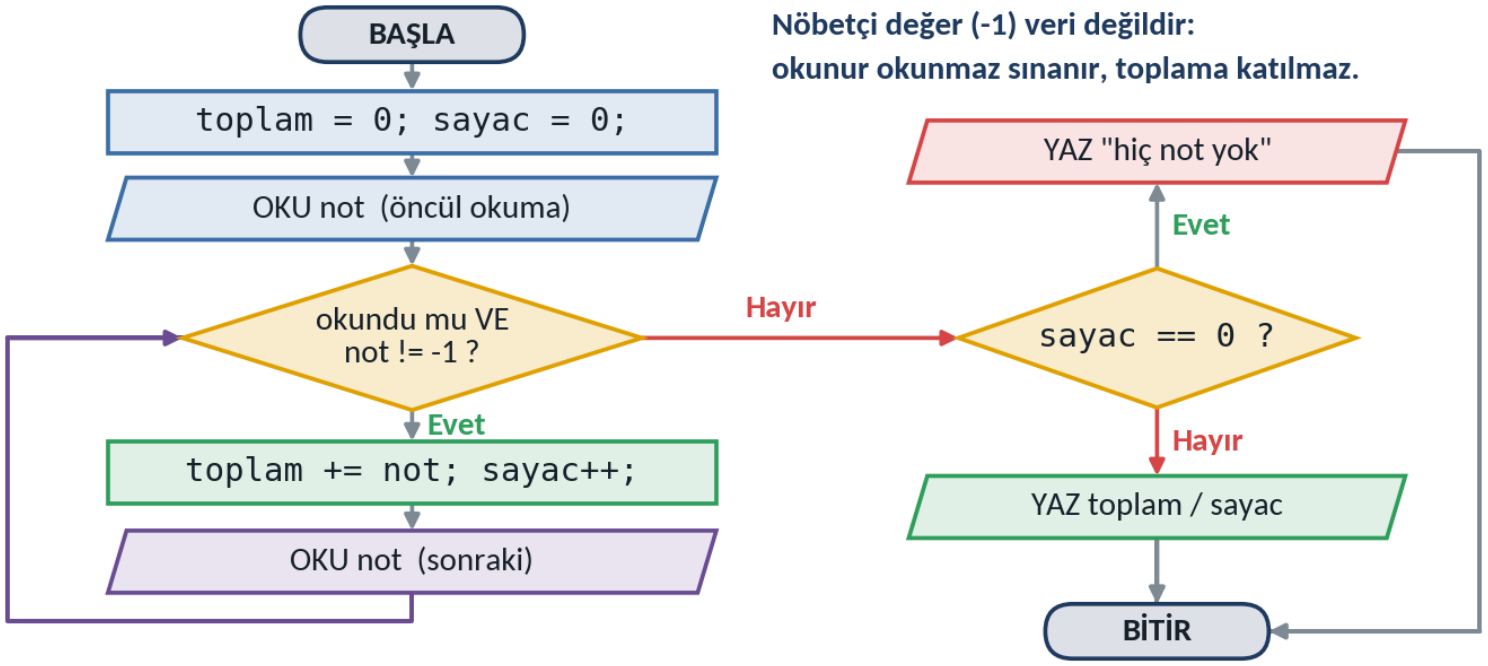
? tur

02

Döngüyü denetlemek ve desenler

sayaç ve nöbetçi değer · break ve continue · altı döngü deseni · izleme tablosu · döngü değişmezi

Sayaç mı, nöbetçi mi?



- **Sayaç kontrollü**: tur sayısı baştan belli ("10 kez", "N öğrenci") → çoğunlukla `for`
- **Nöbetçi değer kontrollü**: veri bitince veri olamayacak bir değer gelir (-1) → `while`
- **Öncül okuma**: ilk değer döngüden **önce** okunur
- Nöbetçi, veri olarak **işlenmez**
- İlk girdi nöbetçiyse **0 tur** → sıfıra bölmeyi önle

break ve continue: akışı kesmek

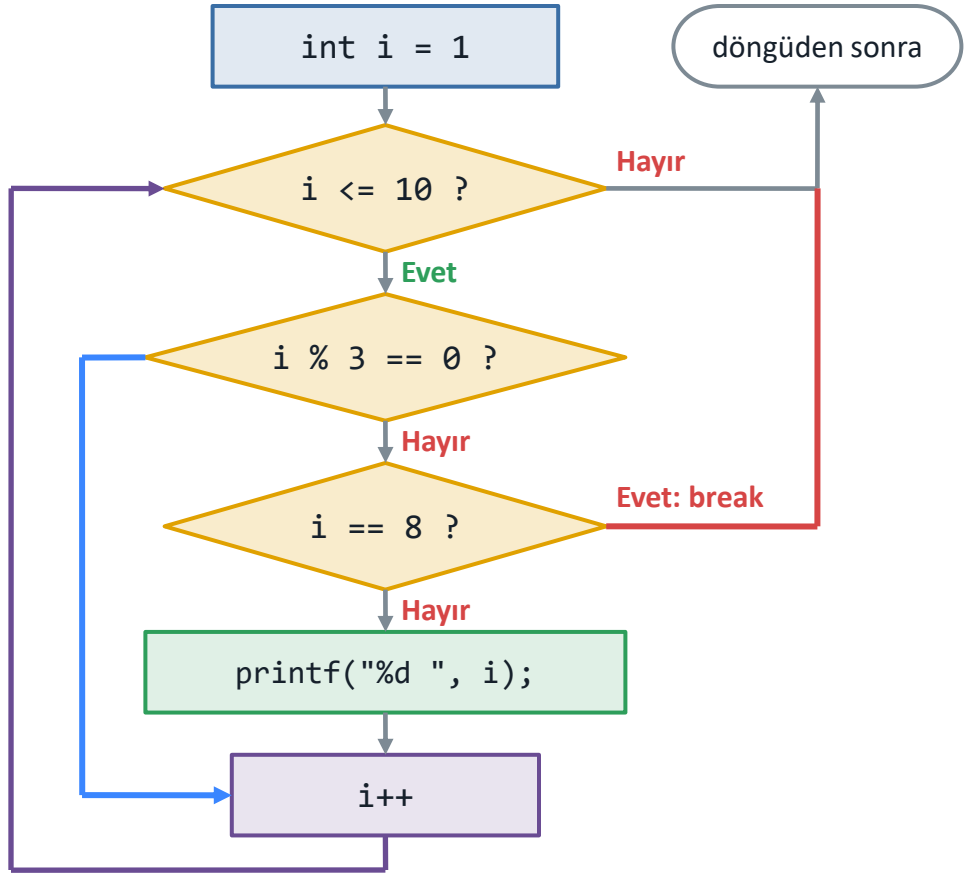
H06_K05_break_continue.c · satır 29–38

```
29      /* birlikte: 3'ün katlarını atla, 8'e gelince dur */
30      for (int i = 1; i <= 10; i++) {
31          if (i % 3 == 0) {
32              continue;
33          }
34          if (i == 8) {
35              break;
36          }
37          printf("%d ", i);
38      }
```

GERÇEK ÇIKTI (TÜM PROGRAM; SON SATIR BU DÖNGÜ)

```
$ ./break_continue
50'den sonraki ilk 7 katı: 56
1 2 4 5 7 8 10
1 2 4 5 7
```

3 ve 6 `continue` ile atlandı, 8'de `break`: `1 2 4 5 7`.



mavi: Evet → continue: turun kalanını atla, i++'ya git

kırmızı: break → döngüden hemen çık

Döngü desenleri (1/2): biriktirme ve bayrak

Toplayıcı

başlangıç: 0 (etkisiz eleman)

K06 · satır 15–18

```
int toplam = 0;
for (int i = 1; i <= N; i++) {
    toplam += i;
}
```

Gerçek sonuç (N = 6): `toplam = 21`

Örnek 2, 7; Mini-OBS sınıf ortalaması

Çarpım biriktirici

başlangıç: 1 (0 değil!)

K06 · satır 29–32

```
long long carpim = 1;
for (int i = 1; i <= N; i++) {
    carpim *= i;
}
```

Gerçek sonuç: `carpim = 720` (6!)

Örnek 5, 6: faktöriyel, üs

Bayrak

başlangıç: 0 (“henüz yok”)

K06 · satır 36–41

```
while (sayi > 0) {
    if (sayi % 10 == 0) {
        sifir_var = 1;
    }
    sayi /= 10;
}
```

sayi = 2026 için gerçek sonuç:
`sifir_var = 1`

“En az bir tane var mı?”; Örnek 13

Döngü desenleri (2/2): sayma, en iyi aday, basamak

Sayaç

başlangıç: 0

K06 · satır 22–26

```
for (int i = 1; i <= N; i++) {
    if (i % 3 == 0) {
        sayac++;
    }
}
```

Gerçek sonuç: `sayac = 2` (3 ve 6)

Örnek 3, 13; Mini-OBS başarılı/şartlı sayısı

En büyük / en küçük

başlangıç: ilk değer (7)

K06 · satır 45–50

```
for (int i = 2; i <= N; i++) {
    int x = (7 * i) % 10;
    if (x > en_buyuk) {
        en_buyuk = x;
    }
}
```

Akış 7 4 1 8 5 2 → `en_buyuk = 8`

Örnek 8; Mini-OBS en yüksek/düşük

Basamak ayrıştırma

başlangıç: m = sayı (2026)

K06 · satır 55–58

```
while (m > 0) {
    printf(" %d", m % 10);
    m /= 10;
}
```

Gerçek çıktı: `6 2 0 2` (sağdan sola)

`% 10` al, `/ 10` at; Örnek 9, 10

İzleme tablosu: her satır bir koşul denetimi

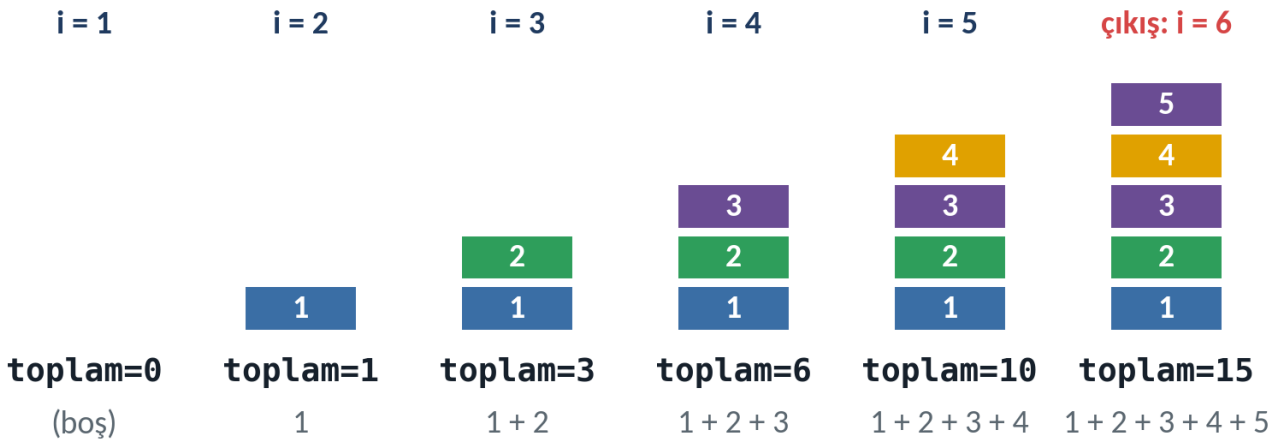
H06_K20_izle_toplam.c · satır 11–16

```
11      int n = 5;
12      int toplam = 0;
13      for (int i = 1; i <= n; i++) {
14          toplam += i;
15      }
16      printf("toplam = %d\n", toplam);
```

Denetim	i	toplam (önce)	koşul	gövde
1	1	0	1 <= 5 → doğru	toplam = 0 + 1 = 1
2	2	1	2 <= 5 → doğru	toplam = 1 + 2 = 3
3	3	3	3 <= 5 → doğru	toplam = 3 + 3 = 6
4	4	6	4 <= 5 → doğru	toplam = 6 + 4 = 10
5	5	10	5 <= 5 → doğru	toplam = 10 + 5 = 15
6	**6**	**15**	**6 <= 5 → yanlış**	**döngüden çık**

Neden biter, neden doğru?

DEĞİŞMEZ: her turun başında toplam = 1 + 2 + … + (i − 1)



İlerleme ölçüsü → bitecek mi?
`n - i + 1` her turda 1 azalır; 0'a inince döngü biter.

Değişmez → doğru mu?
Her turun başında `toplam == 1 + … + (i - 1)`. Çıkışta i = N + 1.

GERÇEK ÇIKTI · H06_K08_DEGİSMEZ.C

```
$ ./degismez
 i  toplam  (i-1)*i/2  değişmez
 1       0          0  korunuyor
 2       1          1  korunuyor
 3       3          3  korunuyor
 4       6          6  korunuyor
 5      10         10  korunuyor
Çıkışta i = 6: toplam = 15 = N(N+1)/2 = 15
```

- Öklid'de `y`, basamakta `sayi` kesin küçülür
- Zune'da 366. günde **hiçbir şey** küçülmez
- Collatz'da ölçü **bilinmiyor**

03

Kaputun altında

Döngü makine düzeyinde nedir? Sayaç nerede yaşar? Derleyici döngüyü formüle çevirebilir mi?

Döngü = geriye doğru koşullu atlama

H06_K10_toplam_fonk.c · satır 11–18

```
11  int toplam_dongu(int n)
12  {
13      int toplam = 0;
14      for (int i = 1; i <= n; i++) {
15          toplam += i;
16      }
17      return toplam;
18  }
```

gcc -O0 -S -masm=intel (kesit)

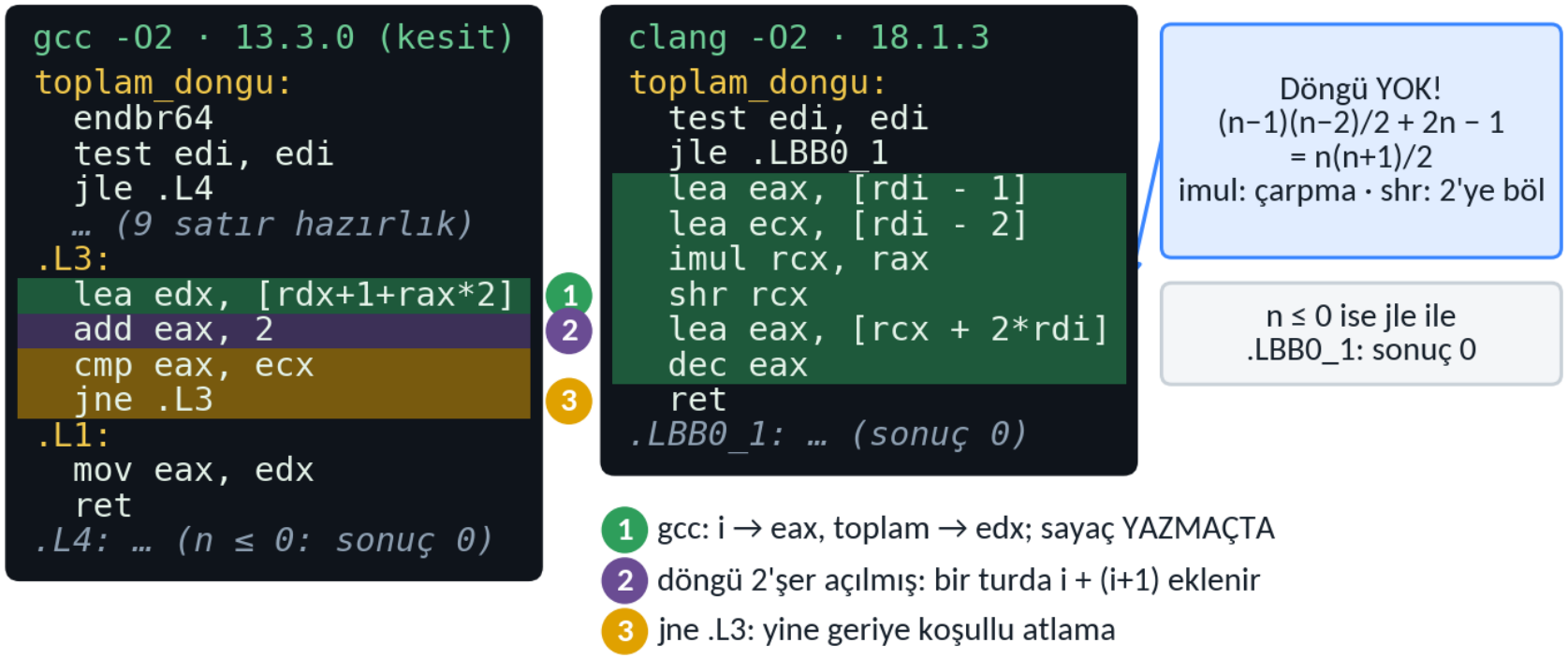
```
    mov DWORD PTR [rbp-8], 0        1
    mov DWORD PTR [rbp-4], 1
    jmp .L2                         2
.L3:
    mov eax, DWORD PTR [rbp-4]
    add DWORD PTR [rbp-8], eax      3
    add DWORD PTR [rbp-4], 1
.L2:
    mov eax, DWORD PTR [rbp-4]
    cmp eax, DWORD PTR [rbp-20]     4
    jle .L3                         5
    mov eax, DWORD PTR [rbp-8]
```

1. toplam = 0, i = 1 (bellekte)
2. önce koşula git (.L2)
3. gövde ve güncelleme: toplam += i; i++
4. i ile n'yi karşılaştır
5. i ≤ n ise .L3'e GERİ atla; değilse çık

- `jmp .L2`: önce **koşula** atla
- `cmp` + `jle .L3`: doğruysa **geri** atla
- Yanlışsa bir alt satıra düş: döngü biter
- `-O0`: `i` ve `toplam` her turda **bellekte** (`[rbp-4]`, `[rbp-8]`)
- Intel sözdizimi: hedef önce; H5'teki AT&T'de sıra ters, `%`/`$` önekli

Gerçek çıktı: kodlar/ciktilar/H06_K10_toplam_fonk_gcc_O0_asm.txt

İyileştirme: sayaç yazmaçta, döngü formülde



Gerçek çıktılar: gcc 13.3 -O2 ve clang 18.1 -O2

- **gcc -O2**: döngü var; `i` → `eax`, `toplam` → `edx` (**yazmaç**)
- 2'şer **açılmış**: `lea edx, [rdx+1+rax*2]`
- **clang -O2**: döngü **yok**; `(n−1)(n−2)/2 + 2n − 1`
- = `n(n+1)/2`: derleyici Gauss'u buldu

Ölçtük: N = 10^9 için 1 + 2 + … + N

2,495 s

gcc **-O0**: sayaç her turda bellekte

0,334 s

gcc **-O2**: yazmaçta, 2'şer açılmış

0,002 s

clang **-O2**: döngü yok, kapalı formül

- Aynı kaynak kod (`H06_K09_milyar_tur.c`); gcc -O2 yaklaşık **7 kat**; clang -O2 **binlerce kat mertebesinde** (süresi süreç başlatmaya yakın: ölçüm sınırında)
- N klavyeden okunur: sonuç derleme anında hesaplanamaz, fark üretilen koddan gelir
- Üç çalıştırmanın sonucu da aynı: `500000000500000000` (gerçek çıktı)

04

Örnek merdiveni

14 örnek, kolaydan zora: her basamak bir yeni öge ekler (★ → ★★★)

Örnek merdiveni: 14 basamak

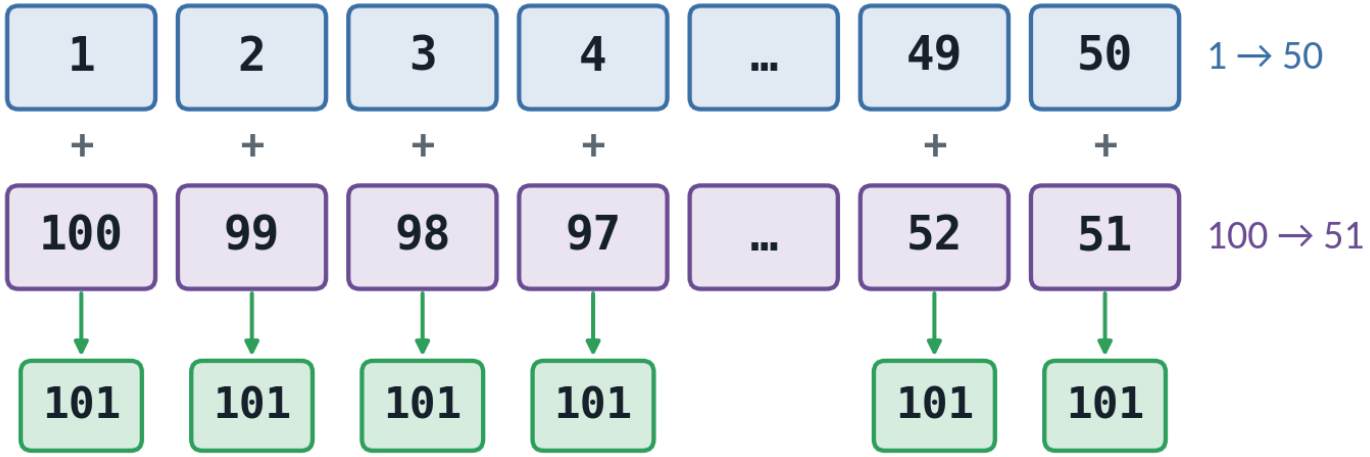
Örnek 1 ★	Örnek 2 ★	Örnek 3 ★	Örnek 4 ★	Örnek 5 ★★	Örnek 6 ★★	Örnek 7 ★★
1'den 10'a kadar yazdırma (üç döngü türüyle)	**1'den N'e kadar toplam**	**Çift sayıların toplamı (iki yol)**	**Bir sayının çarpım tablosu**	**Faktöriyel ve taşma**	**Tekrarlı çarpma ile üs alma**	**Bilinmeyen sayıda notun ortalaması**
while, do-while, for denkliği	Toplayıcı deseni	if ile süzme ile adım 2 ile atlamanın karşılaştırılması	Sayaç kontrollü döngü; biçimli çıktı	Çarpım biriktirici; int → unsigned long long	Döngü ile kuvvet hesabı	Nöbetçi değer (-1) kontrollü döngü

Örnek 8 ★★	Örnek 9 ★★	Örnek 10 ★★	Örnek 11 ★★	Örnek 12 ★★	Örnek 13 ★★	Örnek 14 ★★★
Girilen sayıların en büyüğü ve en küçüğü	**Basamak sayısı ve basamak toplamı**	**Sayıyı ters çevirme ve palindrom sayı**	**Öklid algoritması ile EBOB ve EKOK**	**Geçerli girdi alınana kadar tekrar**	**Sayı tahmin oyunu**	**Collatz dizisi ve adım sayısı**
"Şimdiye kadarki en iyi" deseni	%10 ve /10 döngüsü	Basamaklardan sayı inşası	while ile klasik algoritma (H2'nin kodlanması)	do-while ile girdi doğrulama; scanf dönüş değeri	rand/srand, deneme sayacı, break	Sonlandığı kanıtlanmamış döngü (merak kutusu)

Örnek 2 ★: 1'den N'e toplam (toplayıcı)

H06_02_toplam_n.c · satır 26–31

```
26      int toplam = 0;                 /* toplayıcı: etkisiz eleman 0 */
27      for (int i = 1; i <= n; i++) {
28          toplam += i;                /* toplam = toplam + i */
29      }
30
31      printf("Toplam (1..%d) = %d\n", n, toplam);
```



GERÇEK ÇIKTI · N = 100 VE N = 0

```
$ ./toplam_n
N değerini girin: 100
Toplam (1..100) = 5050

$ ./toplam_n
N değerini girin: 0
Toplam (1..0) = 0
```

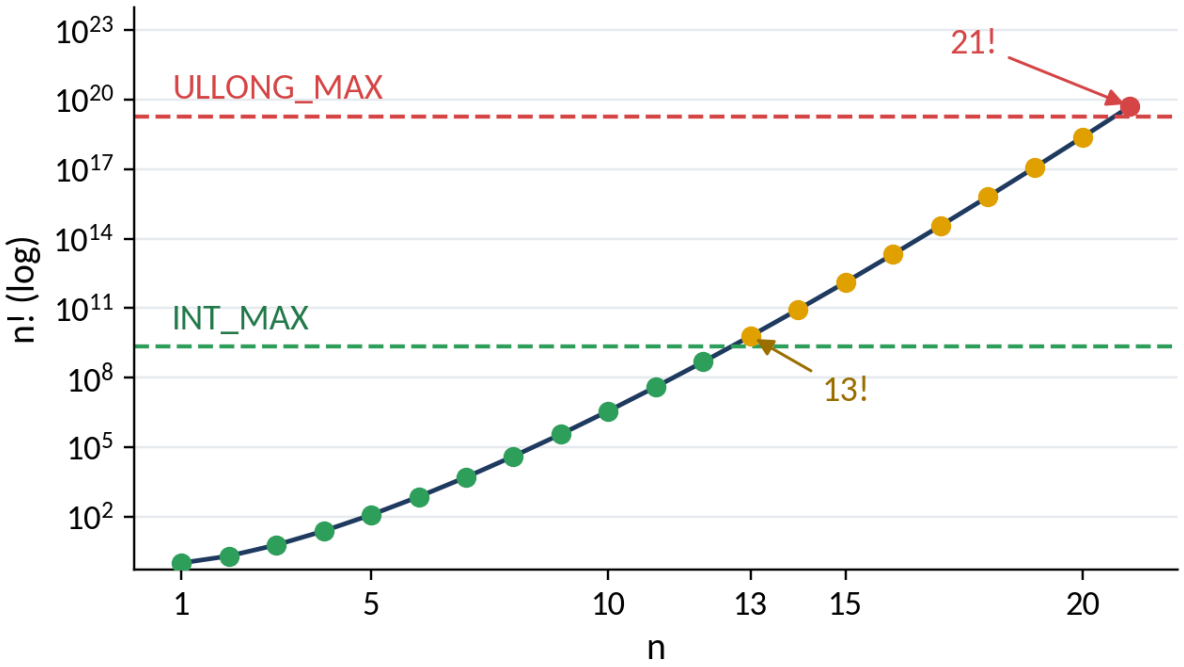
- Döngü: **N tur**; Gauss: 50 × 101, **sabit** iş
- N = 0: döngü **hiç** çalışmaz, boş toplam 0
- N ≤ 65 535 **denetlenir** (EN_BUYUK_N); 65 536 → reddedilir

Örnek 5 ★★: faktöriyel ve taşma

H06_05_faktoriyel_tasma.c · satır 17–20

```
17      for (int n = 1; n <= 21; n++) {
18          /* Taşma denetimi çarpmadan ÖNCE: carpim * n, ULLONG_MAX'ı aşar mı? */
19          int tasacak = carpim > ULLONG_MAX / (unsigned long long)n;
20          carpim *= n;                        /* işaretsiz: 2^64 modunda sarar */
```

Taşma, çarpmadan **önce** sezilir



n	n! (unsigned long long)	int'e sığar mı?
12	479001600	evet
13	6227020800	hayır
19	121645100408832000	hayır
20	2432902008176640000	hayır
21	**14197454024290336768**	**hayır · sarmış!**

Gerçek çıktıdan. 20! sığan son değer; 21! 64 bite sığmaz ve 2^{64} modunda **sarar** (işaretsiz: tanımlı). 13! `int`'e sığmaz.



HATA AVCISI · TANIMSIZ DAVRANIŞ

`int` ile 13!: tanımsız davranış

H06_K07_faktoriyel_int_ub.c · satır 13–17

```
13      int carpim = 1;
14      for (int n = 1; n <= 13; n++) {
15          carpim *= n;                    /* n = 13'te taşar: tanımsız davranış */
16          printf("%2d! = %d\n", n, carpim);
17      }
```

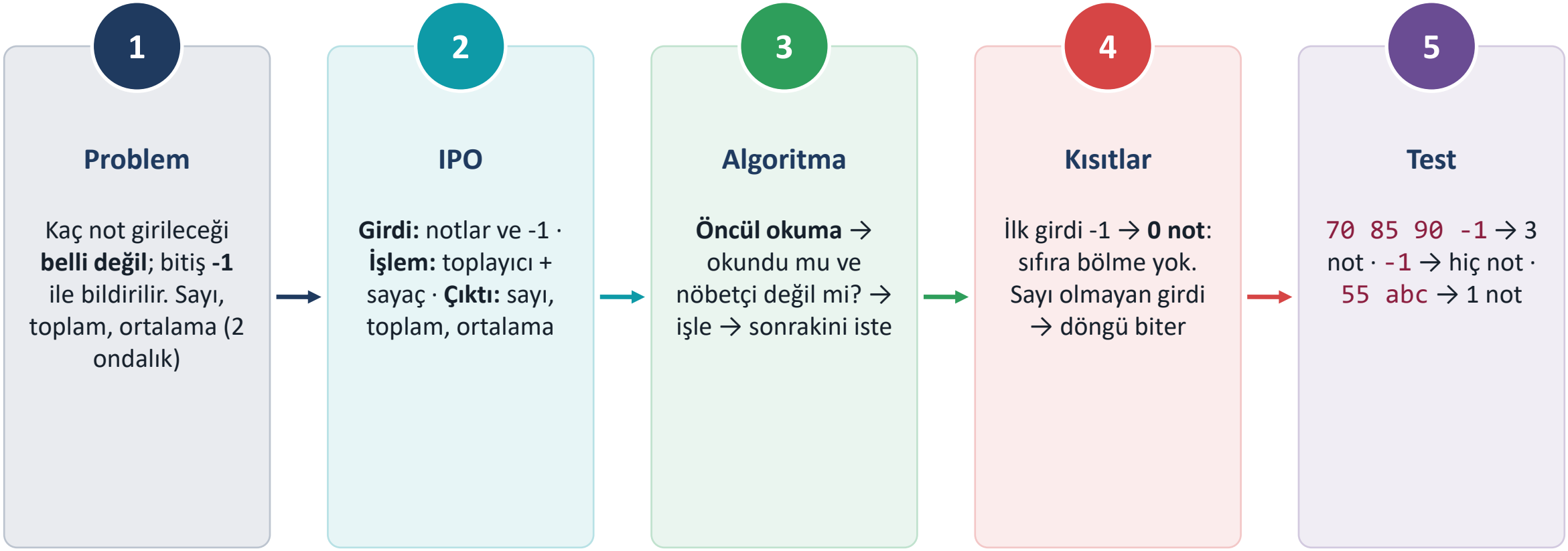
13! = 6 227 020 800 > INT_MAX = 2 147 483 647

UBSan'ın gerçek iletisi (`-fsanitize=undefined` ile derlendi):

```
H06_K07_faktoriyel_int_ub.c:15:16: runtime error: signed integer overflow: 479001600 * 13 cannot be
represented in type 'int'
```

- **İşaretli tamsayı taşması tanımsız davranıştır** (H3): standart bir sonuç öngörmez; iletiden sonraki 13! satırı güvenilir değildir
- Derleyici uyarmaz: değerler **çalışma anında** oluşur. Çözüm: geniş tip (Örnek 5) ya da çarpmadan **önce** sınır denetimi

Amiral örnek — Örnek 7 ★★: bilinmeyen sayıda not



Nöbetçi değer kontrollü döngü: sayısı bilinmeyen verinin standart çözümü

KOD

Örnek 7: nöbetçi döngünün kodu

H06_07_nobetci_ortalama.c · satır 13–24

```
13      const int NOBETCI = -1;             /* "girdi bitti" işareti */
14      int not_degeri;
15      int toplam = 0;                     /* toplayıcı */
16      int sayac = 0;                      /* sayaç     */
17
18      /* öncül okuma: ilk değer döngüden ÖNCE istenir */
19      printf("Not 1 (bitirmek için %d): ", NOBETCI);
20      while (scanf("%d", &not_degeri) == 1 && not_degeri != NOBETCI) {
21          toplam += not_degeri;
22          sayac++;
23          printf("Not %d (bitirmek için %d): ", sayac + 1, NOBETCI);
24      }
```

GERÇEK ÇIKTI · 70 85 90 -1

```
$ ./nobetci_ortalama
Not 1 (bitirmek için -1): 70
Not 2 (bitirmek için -1): 85
Not 3 (bitirmek için -1): 90
Not 4 (bitirmek için -1): -1
3 not, toplam = 245, ortalama = 81.67
```

- Satır 19: **öncül okuma** istemi döngüden önce
- Satır 20: okundu mu **ve** nöbetçi değil mi? (&& kısa devre)
- NOBETCI: adlandırılmış sabit, sihirli sayı yok
- Sonraki istem gövdenin **sonunda**

Örnek 7: izleme tablosu (70, 85, 90, -1)

H06_07_nobetci_ortalama.c · satır 20–24

```
20      while (scanf("%d", &not_degeri) == 1 && not_degeri != NOBETCI) {
21          toplam += not_degeri;
22          sayac++;
23          printf("Not %d (bitirmek için %d): ", sayac + 1, NOBETCI);
24      }
```

Gerçek çıktı: `3 not, toplam = 245, ortalama = 81.67`

Okuma	not_degeri	koşul	toplam (sonra)	sayac (sonra)
1	70	okundu ve -1 değil → doğru	70	1
2	85	okundu ve -1 değil → doğru	155	2
3	90	okundu ve -1 değil → doğru	245	3
4	**-1**	**nöbetçi → yanlış**	**245**	**3**

Örnek 9–10: basamak ayrıştırma ve ters çevirme

tur	sayi	son = sayi % 10	ters = ters * 10 + son	sayi /= 10
1	2026	**6**	**0 * 10 + 6 = 6**	202
2	202	**2**	**6 * 10 + 2 = 62**	20
3	20	**0**	**62 * 10 + 0 = 620**	2
4	2	**2**	**620 * 10 + 2 = 6202**	0

Döngü sayi = 0 olunca durur: 4 basamak → 4 tur. Ters: 6202 (2026 palindrom değil)

- `sayi % 10` → son basamak; `sayi /= 10` → onu **at**
- Örnek 9: `do-while` → 0 da **1 basamak**
- Örnek 10: `ters = ters * 10 + son`
- 2026 → 6202; 1230 → 321 (gerçek çıktı)
- `ters long long`: 1999999999'un tersi `int`'e sığmaz

Örnek 10 (amiral): izleme tablosu (n = 2026)

H06_10_ters_palindrom.c · satır 21–25

```
21      while (sayi > 0) {
22          int son = sayi % 10;          /* 1) son basamağı al        */
23          ters = ters * 10 + son;       /* 2) tersin sağına ekle     */
24          sayi /= 10;                   /* 3) son basamağı at        */
25      }
```

Gerçek çıktı: `Ters: 6202` ve "palindrom değildir"

Tur	sayi (başta)	son	ters (sonra)	sayi (sonra)
1	2026	6	6	202
2	202	2	62	20
3	20	0	620	2
4	2	2	6202	0
5	**0**	**—**	**6202**	**koşul yanlış → çık**

KOD

Örnek 11 ★★: Öklid algoritması (EBOB, EKOK)

H06_11_ebob_ekok.c · satır 20–28

```
20      int x = a, y = b, adim = 0;
21      printf("adım      x      y   x %% y\n");
22      while (y != 0) {                    /* her turda y kesin olarak küçülür */
23          int r = x % y;
24          adim++;
25          printf("%4d %6d %6d %6d\n", adim, x, y, r);
26          x = y;
27          y = r;
28      }
```

GERÇEK ÇIKTI · ÖKLİD'İN KLASİK ÖRNEĞİ (1071, 462)

```
$ ./ebob_ekok
Pozitif iki tamsayı girin: 1071 462
adım      x      y   x % y
   1   1071    462    147
   2    462    147     21
   3    147     21      0
EBOB(1071, 462) = 21
EKOK(1071, 462) = 23562
```

- İlerleme ölçüsü: y her turda **kesin küçülür** (`x % y < y`)
- Sıra önemsiz: 18 48 ilk turda yer değiştirir
- EKOK = a / EBOB · b: **önce böl**, taşma azalır
- Kod her adımı yazdırır: izleme tablosu programın içinde

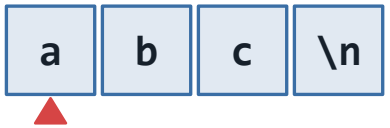
Örnek 12 ★★: geçerli girdi alınana kadar tekrar

H06_12_gecerli_girdi.c · satır 24–30

```
24          if (okunan == 0) {              /* sayı değil: karakterler tamponda */
25              printf("  Sayı bekleniyordu.\n");
26              int c;
27              while ((c = getchar()) != '\n' && c != EOF) {
28                  /* bilinçli boş gövde: satırın kalanını at */
29              }
30              puan = -1;                  /* geçersiz say: döngü sürsün */
```

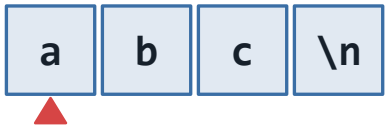
- `scanf` 0 → harfler tamponda **kalır**
- `getchar` döngüsü satırı **boşaltır**

1) Kullanıcı abc yazdı; scanf("%d") ilk karaktere bakar



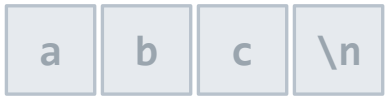
'a' rakam değil → scanf 0 döndürür, hiçbir şey okumaz; 'a' tamponda kalır

2) Dönüşe bakmayan döngü yeniden sorar



scanf yine aynı 'a'yı görür → yine 0 → sonsuz döngü (HA6)

3) Temizleme: getchar satırın kalanını okuyup atar

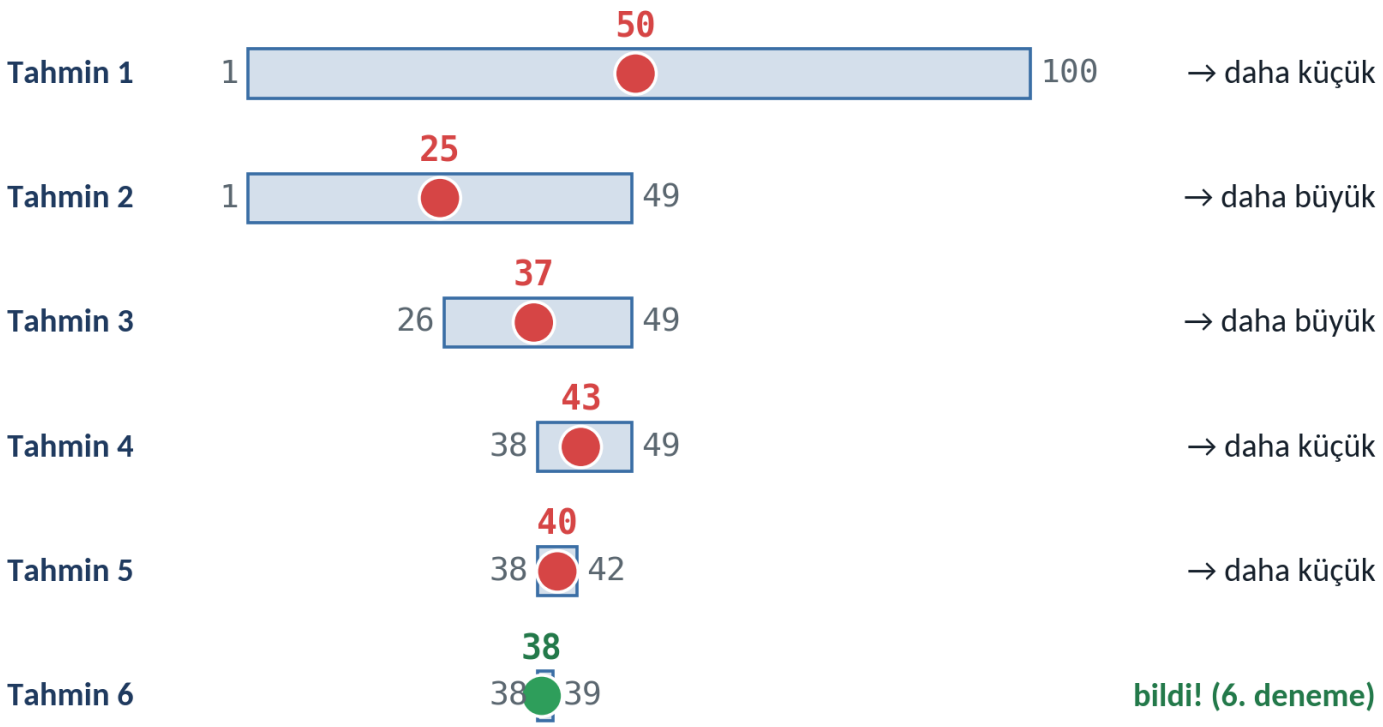


a, b, c ve '\n' tek tek okunur; tampon boşalır, yeni satır beklenir

GERÇEK ÇIKTI · ABC, 150, -5, 85

```
$ ./gecerli_girdi
Puan girin (0-100): abc
  Sayı bekleniyordu.
Puan girin (0-100): 150
  150 aralık dışında.
Puan girin (0-100): -5
  -5 aralık dışında.
Puan girin (0-100): 85
Kabul edilen puan: 85
```

Örnek 13 ★★: sayı tahmin oyunu



Gerçek oturum (H06_13, tohum 2026): her ipucu aralığı yaklaşık yarıya indirir.

- `rand() % 100 + 1` → 1..100
- `RAND_MAX` = 2147483647 (bu makine; standart ≥ 32767)
- `srand(2026)` → hep 38 73 24 70 87: **sözde** rastgele
- `srand((unsigned)time(NULL))`: programın başında **bir kez**
- Aralığı yarılayan en çok **7** tahminde bulur (2^7 = 128 ≥ 100)

KOD

Örnek 13: sayaç + bayrak + break

H06_13_sayi_tahmin.c · satır 42–50

```
42          deneme++;
43          if (tahmin == gizli) {
44              bildi = 1;
45              break;                      /* bulundu: döngüden hemen çık */
46          } else if (tahmin < gizli) {
47              printf("  Daha BÜYÜK bir sayı deneyin.\n");
48          } else {
49              printf("  Daha KÜÇÜK bir sayı deneyin.\n");
50          }
```

GERÇEK ÇIKTI (KESİT; SABIT_TOHUM=2026)

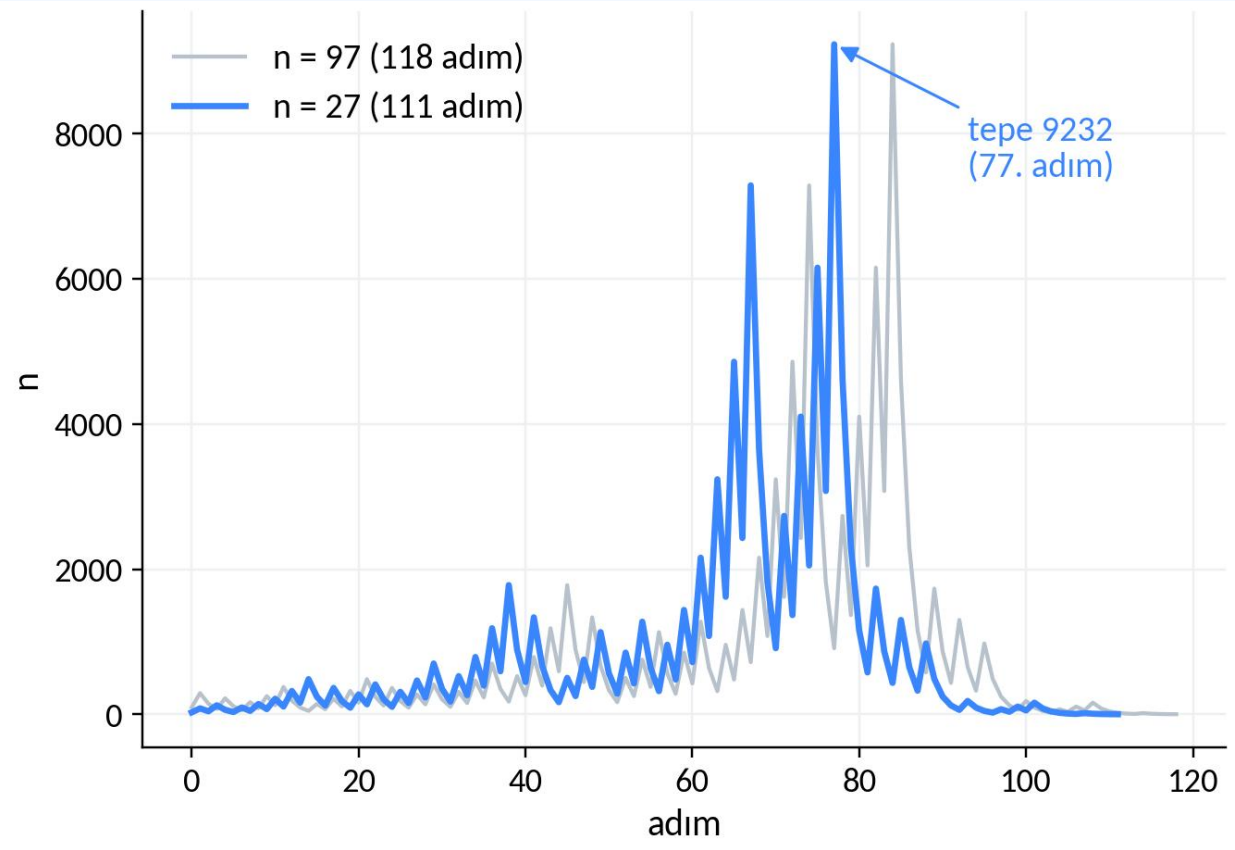
```
$ ./sayi_tahmin
Tahmin 1: 50
  Daha KÜÇÜK bir sayı deneyin.
…
Tahmin 6: 38
Tebrikler! 6 denemede bildiniz.
```

- `deneme` **sayaç**, `bildi` **bayrak**
- Bulununca `break`: döngüden hemen çık
- Sayı olmayan girdide `continue`: hak düşmez

MERAK KUTUSU

Collatz: biteceğini bilmediğimiz döngü

- Kural: `n` çiftse `n / 2`, tekse `3n + 1`; `while (n != 1)`
- 27 → **111 adım**, tepe **9 232** (gerçek çıktı, Örnek 14)
- Her başlangıç için 1'e ulaşılacağı **kanıtlanmadı**; çok büyük değerlere kadar bilgisayarla doğrulandı (Lagarias, 1985)
- İlerleme ölçüsü **bilinmiyor**: "her denediğimizde bitti" bir kanıt değildir
- Kod, `3n + 1` taşmasını işlemden **önce** denetler



05

Hata Avcısı

Sonsuz döngü · gövdesiz for · bir-fark · sıfırlanan toplayıcı · kayan sayaç · scanf tuzağı · Zune

HATA AVCISI

HA1 · Güncellemesi unutulmuş döngü

Hatalı (kasıtlı)

hata_avcisi/H06_HA1_guncelleme_yok.c · satır 12–17

```c
12      int i = 1;
13      printf("1'den 5'e kadar:\n");
14      while (i <= 5) {
15          printf("%d ", i);
16          /* i++ unutuldu: koşul hep doğru kalır */
17      }
```

Düzeltilmiş

hata_avcisi/H06_HA1_guncelleme_yok_duzeltilmis.c · satır 10–15

```c
10      int i = 1;
11      printf("1'den 5'e kadar:\n");
12      while (i <= 5) {
13          printf("%d ", i);
14          i++;                          /* ilerleme: i her turda 1 artar */
15      }
```

- `timeout 2` → çıkış kodu **124**
- 2 saniyede **98 353 152 bayt** “1 ” yazıldı
- Koşul hep doğru: **ilerleme yok**

DÜZELTİLMİŞ · GERÇEK

```
$ ./ha1d
1'den 5'e kadar:
1 2 3 4 5
Bitti.
```

HA2 · for (…); tuzağı: gövdesiz döngü

Hatalı (kasıtlı)

hata_avcisi/H06_HA2_govdesiz_for.c · satır 11–16

```
11      int toplam = 0;
12      int i;
13      for (i = 1; i <= 5; i++);
14      {
15          toplam += i;
16      }
```

Düzeltilmiş

hata_avcisi/H06_HA2_govdesiz_for_duzeltilmis.c · 10–13

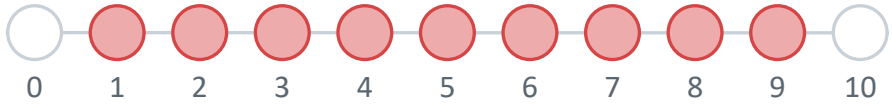
```
10      int toplam = 0;
11      for (int i = 1; i <= 5; i++) {
12          toplam += i;
13      }
```

Gerçek çıktılar: `1..5 toplamı: 6` ↔ `1..5 toplamı: 15` · GCC: `-Wmisleading-indentation` · Clang: `-Wempty-body`

HA3 · HA4 · HA5: sınırlar, sıfırlama ve kayan sayaç

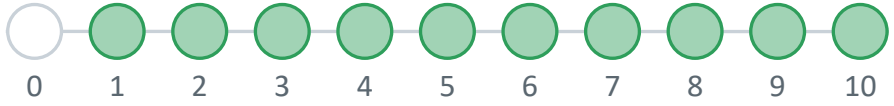
Amaç: 1'den 10'a kadar topla (10 tur). Hangi başlık doğru?

```
for (int i = 1; i < 10; i++)
```



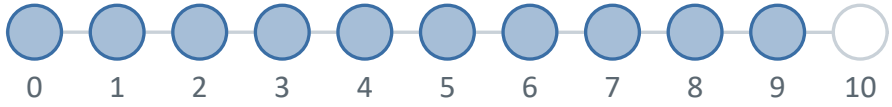
9 tur ✗ (10 yok)

```
for (int i = 1; i <= 10; i++)
```



10 tur ✓

```
for (int i = 0; i < 10; i++)
```



10 tur, ama 0..9

HA3 · bir-fark

`i < N` → gerçek çıktı: toplam **45** (9 tur). `i <= N` ile **55** (10 tur). Tanı: uç değerleri dene.

HA4 · sıfırlanan toplayıcı

`toplam = 0;` döngünün içinde → `Toplam: 100` (doğrusu 340). Başlatma döngüden önce.

HA5 · kayan sayaç

`x != 1.0` hiç yanlış olmaz: 2 s'de 4 825 972 satır. Çare: **tamsayı sayaç**, `x = i / 10.0`. `<` sonsuz döngüyü önler ama **11 tur** döner (Q32).

HATA AVCISI

HA6 · scanf dönüşü yok sayılırsa: tek harf, sonsuz döngü

hata_avcisi/H06_HA6_scanf_harf.c · satır 13–17

```
13      int puan = -1;
14      do {
15          printf("Puan (0-100): ");
16          scanf("%d", &puan);              /* dönüş değeri yok sayılıyor! */
17      } while (puan < 0 || puan > 100);
```

Kök neden: `scanf` 0 döndürür, `abc` tamponda **kalır**; her tur aynı harflere takılır.

GÖSTERİM SÜRÜMÜ (5 TUR SINIRLI) · GERÇEK ÇIKTI

```
$ ./ha6_gosterim
Puan (0-100): abc
[scanf 0 döndürdü, puan = -1]
Puan (0-100): [scanf 0 döndürdü, puan = -1]
Puan (0-100): [scanf 0 döndürdü, puan = -1]
Puan (0-100): [scanf 0 döndürdü, puan = -1]
Puan (0-100): [scanf 0 döndürdü, puan = -1]
5 tur geçti: 'abc' hâlâ tamponda -> sonsuz döngü!
Puan: -1
```

- Saf sürüm: `timeout 2` → 124; 2 s'de **368 156 672 bayt** istem
- **Düzeltme:** dönüşü denetle: EOF → çık, 0 → `getchar` ile satırı at
- Aynı kalıp: Örnek 12, Mini-OBS v0.6
- Dış girdi programı kilitliyorsa: **hizmet engelleme** açığı

HATA AVCISI

HA7 · Zune döngüsü: 366. günde hiçbir şey değişmez

hata_avcisi/H06_HA7_zune_dongusu.c · satır 20–32

```
20      while (gun > 365) {
21          int artik = (yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0;
22          if (artik) {
23              if (gun > 366) {
24                  gun -= 366;
25                  yil += 1;
26              }
27              /* gun == 366 ise hiçbir şey değişmez! */
28          } else {
29              gun -= 365;
30              yil += 1;
31          }
32      }
```

- 2008 artık yıl; 31 Aralık = **gün 366**
- `gun > 365` doğru, `gun > 366` yanlış
- `else` yok → **hiçbir şey** değişmez

SAF SÜRÜM · GERÇEK

```
$ timeout 2 ./ha7; echo "çıkış kodu: $?"
Gün 10593 için yıl hesaplanıyor...
çıkış kodu: 124
```

GÖSTERİM SÜRÜMÜ (TUR SINIRLI) · GERÇEK

```
$ ./ha7_gosterim      # 31 Aralık 2008
tur 28: yil = 2008, gun = 366
tur 29: yil = 2008, gun = 366
…
40 tur geçti, gun hâlâ 366: döngü bitmedi -> sonsuz döngü!
```

HA7 · Düzeltme: her turda ilerleme, tek ve açık çıkış

hata_avcisi/H06_HA7_zune_dongusu_duzeltilmis.c · satır 18–26

```
18      while (1) {
19          int artik = (yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0;
20          int yil_uzunlugu = artik ? 366 : 365;
21          if (gun <= yil_uzunlugu) {
22              break;                      /* gün bu yılın içinde: dur */
23          }
24          gun -= yil_uzunlugu;            /* ilerleme: gun en az 365 azalır */
25          yil++;
26      }
```

- `yil_uzunlugu`: artık yılın iki dalı **birleşti**
- Gün bu yılın içindeyse `break`: tek çıkış noktası
- Her turda `gun` en az 365 **azalır**
- Test: 365, **366**, 367. gün

DÜZELTİLMİŞ SÜRÜM · SINIR TESTLERİ · GERÇEK ÇIKTI

```
$ ./ha7d      # 30 Aralık 2008
1.1.1980'den beri geçen gün (1 = 1.1.1980): 10592
Yıl 2008, yılın 365. günü

$ ./ha7d      # 31 Aralık 2008
1.1.1980'den beri geçen gün (1 = 1.1.1980): 10593
Yıl 2008, yılın 366. günü

$ ./ha7d      # 1 Ocak 2009
1.1.1980'den beri geçen gün (1 = 1.1.1980): 10594
Yıl 2009, yılın 1. günü
```

Mühendis gözüyle: her döngü bitmeli ve doğru sayıda dönmeli

Sonlanma bir tasarım kararıdır

Her döngüde: hangi nicelik **küçülüyor**? Yanıt yoksa **üst sınır** koy (Örnek 13: 7 hak). Gömülü sistemlerde son savunma: **bekçi zamanlayıcı** (watchdog).

Uç değerlerde test

0 tur, 1 tur, çok tur; sınırın hemen öncesi ve sonrası (365, 366, 367. gün). Taşmayı öngör: tipi en büyük sonuca göre seç.

Girdiye güvenme

`scanf` dönüşü denetlenmeyen döngü tek harfle kilitlenir (HA6): dış girdi programı kilitliyorsa bu bir **hizmet engelleme** açığıdır.

Önce doğru ve okunur, sonra hızlı

Yerel iyileştirmeyi derleyici yapar (-O2). Sizin işiniz: doğru döngü, anlamlı adlar, adlandırılmış sabitler (`NOBETCI`) ve **gereksiz turu hiç üretmemek** (Örnek 3).

Mini-OBS v0.6: N öğrenci, akış hâlinde

N'yi oku · `do-while`: N ≥ 1 olana kadar

`for (int i = 1; i <= n; i++)` · her öğrenci için

1. **numara, vize, final oku**
   do-while + scanf dönüşü + tampon temizleme
2. **ortalama ve yuvarlama (v0.4)**
   %40 vize + %60 final; (int)(ortalama + 0.5)
3. **harf ve durum (v0.5, RTEÜ)**
   final < 50 → FF · CC ve üstü başarılı · DC şartlı
4. **akış istatistiği**
   toplayıcı · basarili++ / sartli++ · en yüksek/düşük

SINIF ÖZETİ: ortalama · en yüksek / en düşük (no) · başarılı (CC+) · şartlı (DC)

proje/mini_obs_v0_6.c · satır 166–174

```
166         toplam_ortalama += ortalama;
167         if (i == 1 || ortalama > en_yuksek) {
168             en_yuksek = ortalama;
169             en_yuksek_no = numara;
170         }
171         if (i == 1 || ortalama < en_dusuk) {
172             en_dusuk = ortalama;
173             en_dusuk_no = numara;
174         }
```

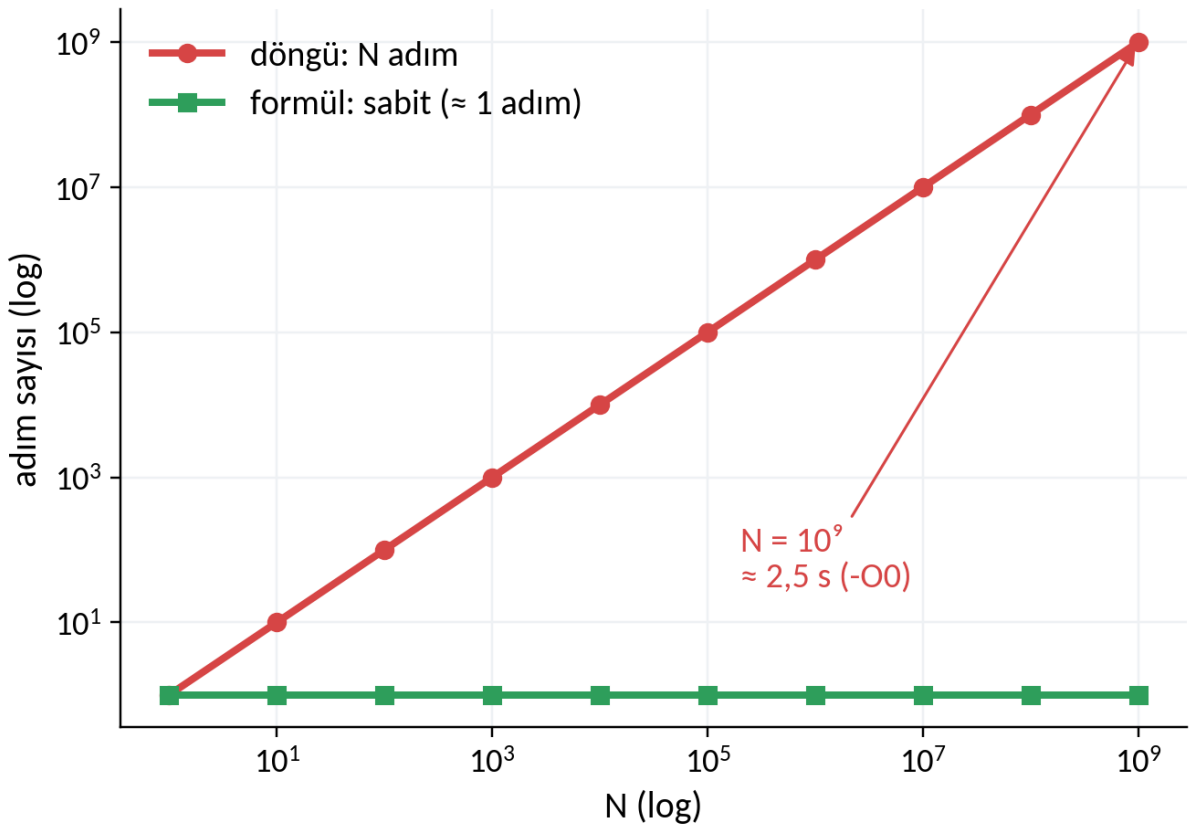
GERÇEK ÇIKTI · T1 (SINIF ÖZETİ)

```
$ ./mini_obs < testler/T1_normal_5_ogrenci.txt
Öğrenci sayısı   : 5
Sınıf ortalaması :  66.48
En yüksek        :  93.00 (2301010047)
En düşük         :  46.80 (2301010046)
Başarılı (CC+)   : 3 / 5
Şartlı (DC)      : 1 / 5
```

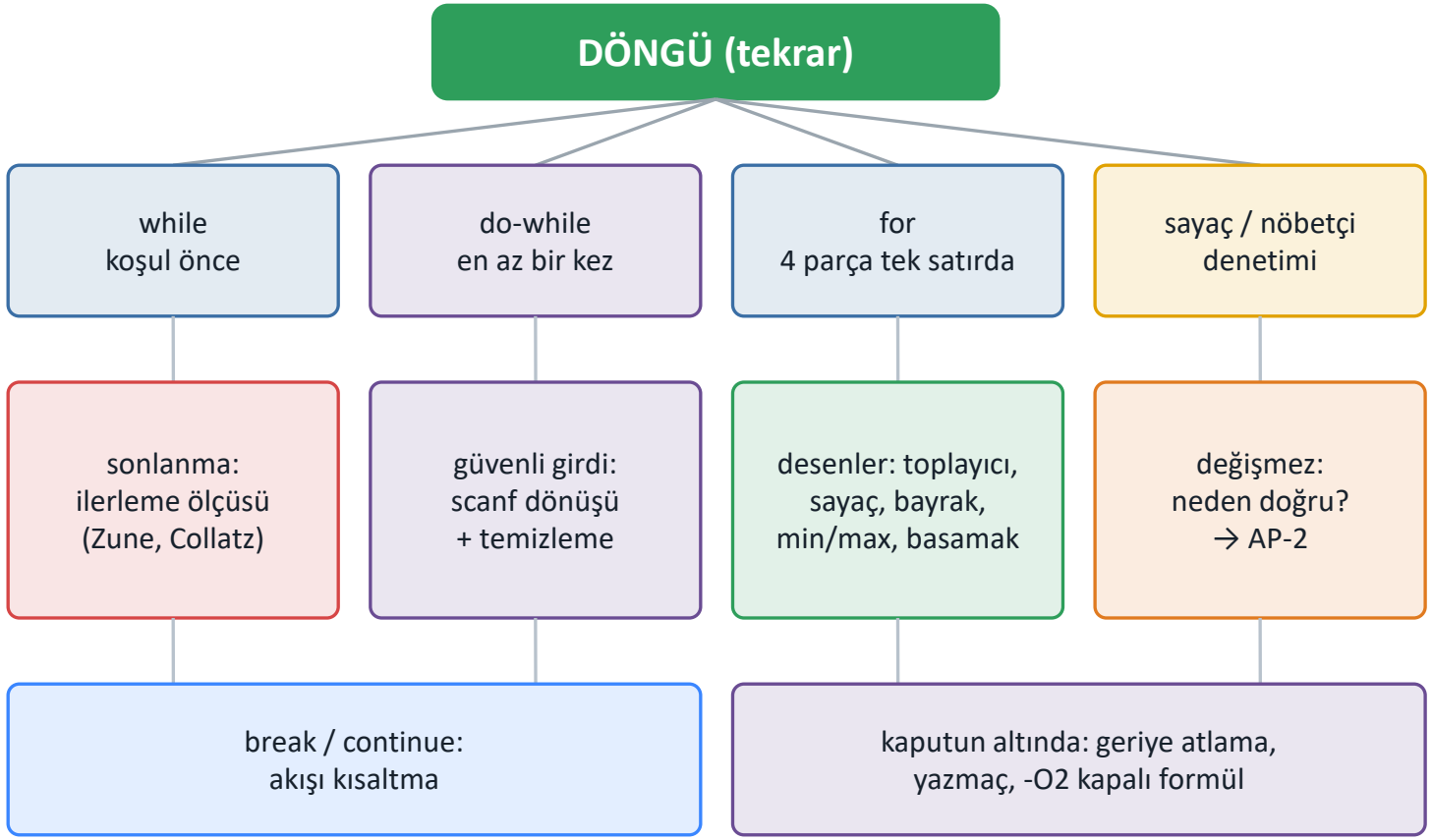
AP-2 KÖPRÜSÜ

Döngü mü, formül mü? Büyüme hızına ilk bakış

- 1 + … + N: döngü **N adım**, formül **sabit** sayıda işlem
- “Girdi büyüdükçe adım sayısı **nasıl büyüyor**?” → AP-2’de algoritma çözümlemesinin temeli
- “Şimdiye kadarki en iyi” deseni → **seçme sıralamasının** iç döngüsü
- Döngü değişmezi → sıralama ve aramanın **doğruluk** gerekçesi



Özet ve Büyük Sorunun yanıtı



- **Nasıl yorulmadan?** Döngü = karşılaştır + geriye atla; 10^9 tur iyileştirmesiz 2,5 s, -O2 ile 0,33 s
- **Durmayı nereden bilir?** Bilmez; biz söyleriz: koşul + her turda sona yaklaştıran güncelleme
- Her döngüye iki soru: **bitecek mi?** (ilerleme ölçüsü) **doğru mu?** (değişmez, izleme, uç değer)
- Zune: ilerleme bir günde durdu → aynı yorulmazlıkla **sonsuza dek**

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1 **Çıktı tahmini:** `for (int i = 10; i > 0; i -= 3)` gövdesinde `printf("%d ", i);` varsa ne yazılır? Döngüden sonra `i`'nin son (yanlış) denetimdeki değeri kaçtır?

2 **Kavram:** while ile do-while arasındaki fark nedir? Kullanıcıdan 0–100 arası puan almak için hangisini seçersiniz, neden?

3 **Hata bul:** `toplam = 0;` satırı döngünün **içine** yazılırsa 70 80 90 100 notlarıyla toplam kaç çıkar? Neden?

KAPANIŞ

Haftaya hazırlık ve kaynaklar

- **Oku:** ders notu Bölüm 3–5; King (2008) Bölüm 6; Deitel ve Deitel (2022) Bölüm 3–4
- **Web:** döngü izleyici, “Kaç kez döner?” testi, sayı tahmin oyunu ve Collatz görselleştiricisi
- **Alıştırmalar:** 6.1–6.6 atölyede; 6.7–6.11 ödev (kodlar/alistirmalar/)
- **Mini-OBS:** v0.6’yı T1–T5 ile çalıştır; kendi test dosyanı ekle
- **H7 önizleme:** iç içe döngüler, asallık testi ve √n, `clock()` ile süre; ara proje: ATM benzetimi
- **Kaynaklar:**
  - Coldewey, D. (2008). The day the Zunes stood still: Update. TechCrunch · Hayes, B. (2009). The Zune bug. bit-player
  - Hayes, B. (2006). Gauss’s day of reckoning. American Scientist, 94(3) · Lagarias, J. C. (1985). The 3x + 1 problem… AMM, 92(1)